ICP133 - Fundamentos de Sistemas de Computação

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
March 31, 2026

Aula 4 - Operações aritméticas em diferentes bases

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
March 31, 2026

Aritmética em Diferentes Bases

- O procedimento para realizar operações aritméticas em qualquer base é aplicar a **tabuada da operação** na base correspondente.
- Na **base 10**, fazemos isso de forma automática devido ao treinamento precoce.
- Em outras bases $(2, 8, 16)$, como não temos isso na nossa cabeça.

Como fazer soma, por exemplo:

1. Trabalhamos coluna por coluna (da direita para a esquerda).
2. Realizamos a soma ou subtração.
3. Se o valor exceder a base, ocorre o **"vai um"** .

Adição Binária: A Tabuada de Bits

Para realizar a adição binária sem converter para a base 10, utilizamos a tabuada da operação. Por ser pequena, sua memorização é simples:

Operando 1	**Operando 2**	**Soma**	**Vai um (Carry)**
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table: Tabuada de adição de bits

- Quando há um "vai um", a próxima coluna passa a ter 3 operandos.
- Aplicamos a soma em etapas: $(1 + 1) + 1 = 10_2 + 1 = 11_2$ (soma 1 e vai 1).

Exemplo de Adição Binária: $11011_2 + 1001_2$

Somamos da direita para a esquerda, respeitando as colunas:

$$
\begin{array}{rcccccc}
\text{(vai um)} & 1 & 1 & 0 & 1 & 1 & \\
 & & 1 & 1 & 0 & 1 & 1_2 \\
+ & & & 1 & 0 & 0 & 1_2 \\
\hline
 & \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{1} & \mathbf{0} & \mathbf{0}_2
\end{array}
$$

Resultado: $11011_2 + 1001_2 = 100100_2$

Nota: em binário, $1+1=10$ *(fica 0 e vai 1) e* $1+1+1=11$ *(fica 1 e vai 1).*

Exercício – Operações na Base 2

Faça as seguintes operações na base 2 **sem converter** os números a serem somados para a base 10 (você pode converter depois de realizar o exercício para conferir o resultado!):

a) $1101_2 + 010_2$

b) $1010_2 + 1010_2$

c) $1111_2 + 1111_2$

d) $101_2 + 1011_2$

Exercício – Operações na Base 2

Gabarito - Faça as seguintes operações na base 2 **sem converter** os números a serem somados para a base 10 (você pode converter depois de realizar o exercício para conferir o resultado!):

a) $1101_2 + 0010_2 = 1111_2$

b) $1010_2 + 1010_2 = 10100_2$

c) $1111_2 + 1111_2 = 11110_2$

d) $0101_2 + 1011_2 = 10000_2$

Adição Octal e Hexadecimal

Exemplo: $172_8 + 30_8$

$$
\begin{array}{rccc}
\text{(carry)} & 1 & & \\
 & 1 & 7 & 2_8 \\
+ & & 3 & 0_8 \\
\hline
 & \mathbf{2} & \mathbf{2} & \mathbf{2}_8
\end{array}
$$

Raciocínio por coluna:

- **Col. 0:** $2 + 0 = 2_8$.
- **Col. 1:** $7 + 3 = 10_{10}$. Como $10 = (1 \times 8) + 2$, temos 12_8. Fica 2 e **vai** 1.
- **Col. 2:** 1 (carry) $+1 = 2_8$.

Exemplo de Adição Hexadecimal: $5D2_{16} + 5A_{16}$

$$
\begin{array}{rccc}
\text{(carry)} & 1 & & \\
 & 5 & D & 2_{16} \\
+ & & 5 & A_{16} \\
\hline
 & \mathbf{6} & \mathbf{2} & \mathbf{C}_{16}
\end{array}
$$

Detalhamento do raciocínio:

Coluna 0: $2_{16} + A_{16} = 2 + 10 = 12_{10}$. Convertendo para hexa: C_{16}.

Coluna 1: $D_{16} + 5_{16} = 13 + 5 = 18_{10}$.
Como $18 = (1 \times 16) + 2$, temos 12_{16}. Fica 2 e **vai** 1.

Coluna 2: 1 (carry) $+5 = 6_{16}$.

Resultado: $5D2_{16} + 5A_{16} = 62C_{16}$

Exercícios: Adição em Bases 8 e 16

Realize as seguintes operações **sem converter os números para a base 10**:

Base 8 (Octal)

a) $5_8 + 6_8$

b) $27_8 + 11_8$

c) $256_8 + 525_8$

Base 16 (Hexadecimal)

a) $503C_{16} + 8_{16}$

b) $503C_{16} + 64_{16}$

Dica: Some coluna por coluna e lembre-se do "vai um" sempre que o valor atingir ou superar a base.

Gabarito: Adição em Bases 8 e 16

Resultados na Base 8:

a) $5_8 + 6_8 = 13_8$

b) $27_8 + 11_8 = 40_8$

c) $256_8 + 525_8 = 1003_8$

Resultados na Base 16:

a) $503C_{16} + 8_{16} = 5044_{16}$

b) $503C_{16} + 64_{16} = 50A0_{16}$

Subtração em Diferentes Bases

- **Subtração Simples:** Quando o dígito do minuendo (cima) é maior ou igual ao do subtraendo (baixo).
- **Subtração com Empréstimo:** Quando o dígito de cima é menor que o de baixo, precisamos "pedir emprestado" à coluna da esquerda.

A Regra do Empréstimo (Borrow)

1. O dígito da esquerda (posição $i + 1$) perde **1**.
2. O dígito atual (posição i) ganha o **valor da base** ($2, 8$ ou 16).

Minuendo	Subtraendo	Diferença	Vem um (Borrow)
0	0	0	0
0	1	**1**	**1 (Empréstimo)**
1	0	1	0
1	1	0	0

Table: Tabuada de subtração de bits

Exemplo 1: Subtração Binária Simples

Neste caso, todos os bits de cima são maiores ou iguais aos de baixo:

$$\begin{array}{rccccc} & 1 & 1 & 0 & 1 & 1_2 \\ - & & 1 & 0 & 0 & 1_2 \\ \hline & \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{1} & \mathbf{0}_2 \end{array}$$

Resultado: $11011_2 - 1001_2 = 10010_2$

Exemplo 2: Subtração com Empréstimo

Quando temos $0 - 1$, pedimos emprestado à esquerda. O 1 vira 0 e a coluna atual ganha 10_2 (que vale 2_{10}).

$$
\begin{array}{rccccc}
\text{(emprestado)} & \text{-1} & \text{-1} & +10 & & \\
 & 1 & 1 & 0 & 1 & 1_2 \\
- & & 1 & 1 & 0 & 1_2 \\
\hline
 & \mathbf{0} & \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{0}_2
\end{array}
$$

Raciocínio:

- **Col. 2:** $0 - 1$ não dá. Pede emprestado: o 1 da Col. 3 vira 0. A Col. 2 vira 10_2. $10_2 - 1 = 1$.
- **Col. 3:** Era 1, virou 0. $0 - 1$ não dá. Pede à Col. 4. O 1 da Col. 4 vira 0. A Col. 3 vira 10_2. $10_2 - 1 = 1$.

Resultado: $11011_2 - 1101_2 = 1110_2$

Exercícios: Subtração Binária

Realize as seguintes operações **sem converter os números para a base 10**.

a) $1101_2 - 100_2$

b) $1010_2 - 101_2$

c) $1000_2 - 111_2$

d) $10011_2 - 101_2$

Gabarito: Subtração Binária

a) $1101_2 - 0100_2 = 1001_2$

b) $1010_2 - 0101_2 = 101_2$

c) $1000_2 - 0111_2 = 1_2$

d) $10011_2 - 00101_2 = 1110_2$

Dica: Para conferir, some o resultado com o subtraendo!

Subtração Octal

Diferente da base 10, o empréstimo aqui vale **8**.

Sem Empréstimo

$$\begin{array}{rccc} & 1 & 7 & 2_8 \\ - & & 3 & 0_8 \\ \hline & \mathbf{1} & \mathbf{4} & \mathbf{2}_8 \end{array}$$

Sem empréstimo, a sequência de símbolos é idêntica à base 10.

Com Empréstimo

$$\begin{array}{rccc} \text{(emprestado)} & -1 & +10_8 & \\ & 1 & 2 & 2_8 \\ - & & 7 & 0_8 \\ \hline & \mathbf{0} & \mathbf{3} & \mathbf{2}_8 \end{array}$$

Raciocínio Col. 1:
$2_8 < 7_8 \rightarrow$ pede emprestado.
$(2+8)_{10} = 10_{10}$.
$10 - 7 = 3$. Resultado: 3_8.

Subtração Hexadecimal: Exemplo $5D2_{16} - 5A_{16}$

(emprestado)		-1	$+10_{16}$
	5	D	2_{16}
-		5	A_{16}
	5	**7**	$\mathbf{8}_{16}$

Detalhamento:

- **Col. 0:** $2 < A(10)$. Pede emprestado: $(2 + 16)_{10} = 18_{10}$. $18 - 10 = 8$. Resultado: 8_{16}.
- **Col. 1:** O $D(13)$ emprestou 1, virou $C(12)$. $12 - 5 = 7$. Resultado: 7_{16}.
- **Col. 2:** $5 - 0 = 5$.

Exercícios: Subtração Octal e Hexadecimal

Realize as operações diretamente na base indicada:

Base 8 (Octal)

a) $254_8 - 123_8$

b) $56_8 - 37_8$

Base 16 (Hexadecimal)

a) $503C_{16} - 40_{16}$

b) $50EA_{16} - 503C_{16}$

Dica: Converta o símbolo para decimal apenas para fazer a conta da coluna, depois volte para a base original.

Gabarito: Subtração Octal e Hexadecimal

Respostas Base 8:

a) $254_8 - 123_8 = 131_8$

b) $56_8 - 37_8 = 17_8$

Respostas Base 16:

a) $503C_{16} - 40_{16} = 4FFC_{16}$

b) $50EA_{16} - 503C_{16} = AE_{16}$

**No item (a) de Hexa, o 0 da posição 503C precisou pedir emprestado ao 5, tornando-se 15 (F) após passar o empréstimo adiante.*

Multiplicação em Qualquer Base

- O processo segue a mesma lógica da base 10: multiplicamos cada dígito do segundo operando por todos os dígitos do primeiro.
- Cada nova linha de produto parcial é deslocada uma posição à esquerda.
- O resultado final é a **soma** de todos os produtos parciais.

Multiplicação Binária

No mundo binário, a operação é simplificada:

- Se o multiplicador é **1**: Copiamos o primeiro operando.
- Se o multiplicador é **0**: A linha será composta apenas por zeros.

Multiplicação Binária: A Regra

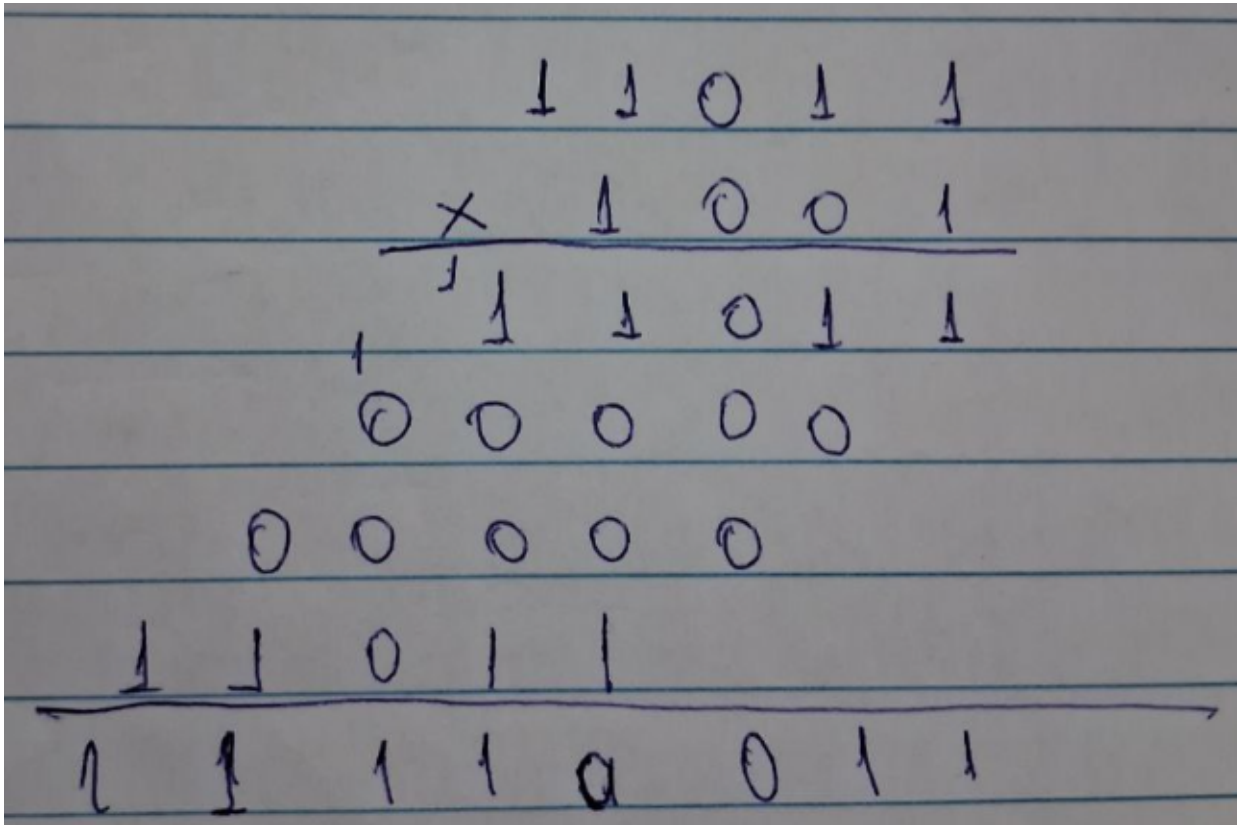
A multiplicação binária é baseada em uma tabuada muito simples. Na prática, multiplicamos o primeiro operando bit a bit pelos dígitos do segundo operando.

Primeiro número	**Segundo número**	**Produto**
0	0	0
0	1	0
1	0	0
1	1	1

O que observar:

- Multiplicar por **1** é apenas repetir o número.
- Multiplicar por **0** resulta em zero.
- Lembre-se de deslocar cada linha de produto parcial à esquerda.

Exemplo de Multiplicação: $11011_2 \times 1001_2$



Dica: Após gerar as linhas de produto, aplique a regra da **soma binária** com atenção aos "vai um" (carries) gerados.

Multiplicação Octal e Hexadecimal

- O processo exige converter os produtos parciais para decimal e depois reconvertê-los para a base original.
- **Importante:** O "vai um" (carry) aqui pode ser qualquer valor, dependendo de quantas vezes a base cabe no resultado do produto.

Exemplo Octal: $172_8 \times 30_8$

$$
\begin{array}{rrrrrl}
 & 1 & 7 & 2 & {}_8 & \\
\times & & 3 & 0 & {}_8 & \\
\hline
 & 0 & 0 & 0 & & (\times 0) \\
+\,5 & 5 & 6 & & & (\times 3) \\
\hline
\mathbf{5} & \mathbf{5} & \mathbf{6} & \mathbf{0} & {}_8 &
\end{array}
$$

Raciocínio ($3 \times 172_8$):

- $3 \times 2 = 6_8$.
- $3 \times 7 = 21_{10} \rightarrow (2 \times 8) + 5 = 25_8$. Fica **5** e vão **2**.
- $3 \times 1 = 3$. Com os 2 que foram: $3 + 2 = 5_8$.

Exemplo Hexadecimal: $5D2_{16} \times 51_{16}$

$$
\begin{array}{rccccl}
 & & 5 & D & 2 & {}_{16} \\
\times & & & 5 & 1 & {}_{16} \\
\hline
 & & 5 & D & 2 & (\text{x}1) \\
+\,1 & D & 1 & A & & (\text{x}5) \\
\hline
\mathbf{1} & \mathbf{D} & \mathbf{7} & \mathbf{7} & \mathbf{2} & {}_{16}
\end{array}
$$

Destaque do cálculo (x5):

- $5 \times 2 = 10_{10} = \mathbf{A_{16}}$.
- $5 \times D(13) = 65_{10} \rightarrow (4 \times 16) + 1 = \mathbf{41_{16}}$. Fica **1** e vão **4**.
- $5 \times 5 = 25_{10}$. Com o carry: $25 + 4 = 29_{10} \rightarrow (1 \times 16) + 13 = \mathbf{1D_{16}}$.

Exercícios: Multiplicação em Bases Diversas

Realize as operações diretamente na base indicada:

Binário e Octal

a) $1010_2 \times 11_2$

b) $1100_2 \times 101_2$

c) $123_8 \times 43_8$

Hexadecimal

d) $1A2_{16} \times 21_{16}$

Gabarito: Multiplicação

Respostas:

a) $1010_2 \times 11_2 = 11110_2$

b) $1100_2 \times 101_2 = 111100_2$

c) $123_8 \times 43_8 = 5531_8$

d) $1A2_{16} \times 21_{16} = 35E2_{16}$

Divisão Binária

- O algoritmo é idêntico ao da divisão decimal (método da chave).
- **Vantagem:** Cada bit do quociente só pode ser **0** ou **1**.
  - **1:** Se o dividendo parcial for maior ou igual ao divisor.
  - **0:** Se o dividendo parcial for menor que o divisor.
- É a operação base para implementações de hardware em camadas próximas ao processador.

Exemplo: $100100_2 \div 111_2$

			'	'	'			
1	0	0	1	0	0	1	1	1
	1	1	1			1	0	1
	0	1	0	0	0			
			1	1	1			
		0	0	0	1			

Figure: Processo de divisão bit a bit

Passo a Passo da Divisão: $100100_2 \div 111_2$

1. **Busca:** Percorremos o dividendo da esquerda para a direita até encontrar um valor $\geq$ ao divisor (111_2).
2. **Primeiro Bit:** Como $1001_2 \geq 111_2$, o primeiro bit do quociente é **1**.
3. **Subtração:** $1001_2 - 111_2 = 0010_2$ (resto parcial).
4. **Baixar Bit:** "Baixamos" o próximo 0. O novo dividendo parcial é 100_2.
5. **Bit Zero:** Como $100_2 < 111_2$, o próximo bit do quociente é **0**.
6. **Novo Bit:** Baixamos o último 0. Agora temos 1000_2.
7. **Bit Um:** Como $1000_2 \geq 111_2$, o bit final do quociente é **1**.
8. **Resto Final:** $1000_2 - 111_2 = 001_2$.

Resultado: Quociente $= 101_2$ — Resto $= 1_2$

Casos Particulares: Multiplicação e Divisão por B^n

Operações por potências da base (B^n) são simplificadas pelo **deslocamento da vírgula**.

- **Multiplicação ($\times B^n$):** Desloca-se a vírgula para a **direita** (acrescentando zeros se necessário).
- **Divisão ($\div B^n$):** Desloca-se a vírgula para a **esquerda**.

Analogia no Sistema Decimal

- $12 \times 10^1 = 120$ (acrescenta um 0)
- $120 \div 10^1 = 12$ (remove um 0 ou desloca a vírgula)

Essa lógica é idêntica para a base 2, substituindo 10 por 2.

Multiplicação por Potência de 2 (2^n)

Para multiplicar um binário por 2^n, basta acrescentar n zeros ao final do número.

Exemplos Práticos

- $11_2 \times 2^1$: $3_{10} \times 2 = 6_{10} \rightarrow \mathbf{110_2}$
- $101_2 \times 2^1$: $5_{10} \times 2 = 10_{10} \rightarrow \mathbf{1010_2}$

- **Exercício 18:** Realize as multiplicações na base 2:
  a) $111 \times 2^1 = \ldots$
  b) $110 \times 2^2 = \ldots$
  c) $11 \times 2^3 = \ldots$

Divisão por Potência de 2 (2^n)

Para dividir um binário por 2^n, deslocamos a vírgula n casas para a **esquerda**.

Exemplos Práticos

- $110_2 \div 2^1$: $6_{10} \div 2 = 3_{10} \rightarrow \mathbf{11_2}$
- $1000_2 \div 2^2$: $8_{10} \div 4 = 2_{10} \rightarrow \mathbf{10_2}$
- $111_2 \div 2^1$: $7_{10} \div 2 = 3,5_{10} \rightarrow \mathbf{11,1_2}$

Prova Real ($11,1_2$)

$(1 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) = 2 + 1 + 0,5 = \mathbf{3,5_{10}}$

Exercício 19: Divisão na Base 2

Efetue as divisões deslocando a vírgula conforme a potência de 2:

a) $1010 \div 2^1$

b) $1100 \div 2^2$

c) $11000 \div 2^3$

d) $101 \div 2^1$

Dica

Se o número original não tiver zeros suficientes para "cortar", a vírgula (que estava implícita à direita) deve aparecer e caminhar para a esquerda.

Tabela de Potências de 2 (2^0 a 2^{32})

n	2^n	**Na Base 2**
0	1	1
1	2	10
2	4	100
3	8	1000
4	16	10000
5	32	100000
6	64	1000000
7	128	10000000
8	256	100000000
9	512	1000000000
10	1.024	10000000000

n	2^n
11	2.048
12	4.096
13	8.192
14	16.384
15	32.768
16	65.536
17	131.072
18	262.144
19	524.288
20	1.048.576
21	2.097.152

n	2^n
22	4.194.304
23	8.388.608
24	16.777.216
25	33.554.432
26	67.108.864
27	134.217.728
28	268.435.456
29	536.870.912
30	1.073.741.824
31	2.147.483.648
32	4.294.967.296

Aula 5 - Precisão Finita

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
March 31, 2026

Números de Precisão Finita

- **No papel:** Usamos quantos dígitos forem necessários.
- **No Computador:** O espaço (memória) é limitado e definido no projeto do hardware.

Definição

Computadores lidam com uma quantidade **fixa** de dígitos. Isso nos força a trabalhar com um conjunto limitado de valores representáveis.

O que estudaremos nesta seção:

- Propriedades algébricas na precisão finita.
- Cálculo da quantidade de valores representáveis na base B.
- Representação de números inteiros com sinal.

Limitações da Precisão Finita

Considere um sistema hipotético que usa apenas **3 dígitos decimais** (000 a 999):

- **Valores impossíveis de representar:**
  - Maiores que 999 (Ex: 1000, 1500).
  - Menores que 0 (Ex: -1, -50).
  - Frações e números irracionais (Ex: 0,5 ou π).

Quebra de Propriedades

Diferente da matemática tradicional, os números de precisão finita **não possuem a propriedade de fechamento**.

A Propriedade de Fechamento

- **Na Matemática (Conjunto $\mathbb{Z}$):** Para qualquer i, j, os resultados de $i + j, i - j$ e $i \times j$ também são inteiros.
- **No Computador:** O resultado de uma operação pode "escapar" da faixa permitida.

Exemplos de Falha (Sistema de 3 dígitos)

- **Soma:** $900 + 500 = 1400$ *(Lupa: Cabe apenas 400?)*
- **Subtração:** $1 - 2 = -1$ *(Não há sinal negativo)*
- **Multiplicação:** $100 \times 10 = 1000$ *(Estourou o limite)*

O Erro de Overflow

OVERFLOW

- **O que é:** Ocorre quando o resultado de uma operação aritmética está fora da faixa de valores que o sistema consegue representar.
- **Analogia:** É como um hodômetro de carro que chega em 999.999km e volta para 000.000km.

Impacto no Software

Se o programador não previr o overflow, o computador pode produzir resultados bizarros (como somar dois números positivos e obter um número negativo ou um valor muito pequeno).

Propriedades Algébricas na Precisão Finita

Diferente da Matemática Clássica, onde os conjuntos numéricos são infinitos, o hardware precisa lidar com limites físicos. Isso afeta como as propriedades algébricas se comportam.

Detecção de Erro

No caso de **overflow**, o hardware (CPU) é projetado para identificar o estouro de capacidade e emitir um aviso (*flag*) para o sistema operacional.

Vamos analisar as três propriedades fundamentais:

1. **Comutatividade**
2. **Associatividade**
3. **Distributividade**

Comutatividade e Associatividade

- **Comutatividade:** Mantém-se inalterada.
  - $a + b = b + a$
  - $a \times b = b \times a$
  - A ordem dos operandos não altera o resultado, mesmo que ocorra overflow.

- **Associatividade:** É satisfeita, **desde que o resultado final seja válido**.
  - $(a + b) + c = a + (b + c)$
  - $(a \times b) \times c = a \times (b \times c)$

Associatividade

Se o resultado final estiver dentro da capacidade de representação, o valor será o mesmo. Se houver overflow, ele ocorrerá independentemente da ordem de execução, tornando o resultado inválido em ambos os casos.

Distributividade na Precisão Finita

A última propriedade fundamental é a **Distributividade** da multiplicação sobre a adição:

Fórmula

$$a \times (b + c) = (a \times b) + (a \times c)$$

- **No Computador:** A propriedade se mantém válida para números inteiros.
- **A Ressalva:** Assim como na associatividade, a igualdade é garantida se:
  1. O resultado final for **válido** (dentro da faixa representável).
  2. **OU** se ambos os caminhos de cálculo resultarem em um valor **inválido** (Overflow).

Conclusão das Propriedades

Para **Inteiros**, a Álgebra de Precisão Finita tenta mimetizar a Álgebra Clássica. O maior "vilão" não é a quebra das regras de ordem, mas sim o **estouro do limite de bits**.

Por que as Propriedades importam?

Mesmo sem decorar os nomes, usamos essas regras para simplificar cálculos no dia a dia:

- **Comutatividade + Associatividade:** Permitem somar uma lista de números em qualquer ordem (ex: agrupar números que somam 10 para facilitar a conta de cabeça).
- **Distributividade:** É a base para **colocar fatores em evidência** em expressões algébricas.

No Computador

Essas manipulações são usadas pelos **Compiladores** para otimizar o código, reorganizando as operações para que o processador as execute de forma mais rápida.

Quantos números cabem no sistema?

Para entender a capacidade de um hardware, precisamos resolver um problema de contagem:

O Problema

Dados B símbolos diferentes, de quantas maneiras podemos preencher n posições distintas (onde a ordem importa e símbolos podem se repetir)?

- **Posição 0:** B opções.
- **Posição 1:** B opções.
- ...
- **Posição** $n-1$**:** B opções.

$$\textbf{Total de Combinações} = B^n$$

Princípio Multiplicativo: B^n

A fórmula B^n surge do princípio fundamental da contagem:

- Com **1 posição**: B possibilidades.
- Com **2 posições**: $B \times B = B^2$ possibilidades.
- Com **3 posições**: $B \times B \times B = B^3$ possibilidades.

Exemplo em Binário ($B = 2$)

Se um computador usa **8 bits** ($n = 8$):

$$2^8 = 256 \text{ combinações diferentes}$$

Se ele usa 10 bits:

$$2^{10} = 1.024 \text{ combinações}$$

Exercícios: Capacidade de Representação

Utilizando a fórmula B^n, determine a quantidade de combinações possíveis para cada caso:

a) Quantos números diferentes podem ser expressos usando **2 dígitos** na base **10**?

b) Quantos números diferentes podem ser expressos usando **5 dígitos** na base **2**?

c) Quantos números diferentes podem ser expressos usando **32 dígitos** na base **2**?

Dica: Lembre-se da nossa tabela de potências de 2 vista anteriormente!

Gabarito Comentado

a) Base 10, $n = 2$**:**

$$10^2 = \mathbf{100} \text{ combinações}$$

(Os números de 00 a 99)

b) Base 2, $n = 5$**:**

$$2^5 = \mathbf{32} \text{ combinações}$$

(Os números binários de 00000 a 11111)

c) Base 2, $n = 32$**:**

$$2^{32} = \mathbf{4.294.967.296} \text{ combinações}$$

(Aproximadamente 4,3 bilhões de valores diferentes)

A quantidade de bits (n) define o "tamanho do universo" de números que podemos contar. Se o resultado de uma conta for o número $2^{32} + 1$, ele não "caberá" no universo que so temos 32 bits, gerando o **Overflow**.

Como representar o sinal?

Até agora, vimos números "sem sinal" (unsigned), onde todos os bits representam magnitude. Para incluir números negativos, precisamos de convenções:

- **Números sem sinal:** Faixa de 0 a $B^n - 1$.
- **Números com sinal:** Três esquemas principais:
  1. **Sinal-Magnitude:** O bit mais à esquerda indica o sinal ($0 = +, 1 = -$).
  2. **Excesso de 2^{n-1}:** O valor é deslocado por uma constante.
  3. **Complementos (1 e 2):** O bit de sinal faz parte do valor aritmético.

O problema do Zero

Como n bits geram uma quantidade par de combinações (2^n), ao usarmos uma para o zero, sobra um número ímpar de padrões para dividir entre positivos e negativos.

O Esquema Sinal-Magnitude

É a forma mais intuitiva, semelhante ao que fazemos no papel:

b_{n-1}	b_{n-2}	$\dots$	b_1	b_0
Sinal	**Magnitude (Valor Absoluto)**			

- **Bit de Sinal (b_{n-1}):** $0 \to$ Positivo $(+)$ — $1 \to$ Negativo $(-)$.
- **Magnitude:** Representada pelos $n-1$ bits restantes.
- **Faixa de Valores:** De $-(B^{n-1}-1)$ até $B^{n-1}-1$.

O Problema do "Zero Duplo"

Neste sistema, o zero possui duas representações:

- $+0$ (ex: 0000) e -0 (ex: 1000).

Isso desperdiça uma combinação e complica a lógica do hardware.

Exemplos e Exercício (Sinal-Magnitude)

Considere $n = 4$ bits:

- $0011_2 \rightarrow +3_{10}$
- $1011_2 \rightarrow -3_{10}$
- $1000_2 = 0000_2 = 0$

Converta os decimais para binário usando **5 bits** em **Sinal-Magnitude**:

a) -15

b) -1

c) 0 (apresente as duas formas)

d) 10

Gabarito: Exercício 21

Para $n = 5$ bits, o primeiro bit é o sinal e os outros 4 são a magnitude:

a) $-15 \rightarrow$ Sinal: $1 \rightarrow$ Magnitude (15_{10}): $1111_2 \rightarrow$ **11111**

b) $-1 \rightarrow$ Sinal: $1 \rightarrow$ Magnitude (1_{10}): $0001_2 \rightarrow$ **10001**

c) $0 \rightarrow$ **00000** ($+0$) e **10000** (-0)

d) $10 \rightarrow$ Sinal: $0 \rightarrow$ Magnitude (10_{10}): $1010_2 \rightarrow$ **01010**

Reflexão

Note que em Sinal-Magnitude, para tornar um número negativo, basta "ligar" o bit mais significativo. O restante do número permanece idêntico.

O Esquema de Excesso de 2^{n-1}

Neste sistema, não "reservamos" um bit de sinal isolado. Em vez disso, somamos um valor fixo ao número real para deslocá-lo para a faixa positiva.

Funcionamento

Para n bits, o excesso é 2^{n-1}.

$$\text{Representação Binária} = \text{Valor Decimal} + 2^{n-1}$$

- **Inversão de Lógica:** Ao contrário do Sinal-Magnitude, aqui:
  - Números que começam com **0** são **Negativos**.
  - Números que começam com **1** são **Positivos** (ou zero).
- **Vantagem:** Mantém a **ordem crescente** dos bits igual à ordem dos valores decimais.
- **Zero Único:** O valor 0 é representado pelo padrão $100...0_2$.

Exemplo: $n = 4$ (Excesso de $2^3 = 8$)

Com 4 bits, somamos 8 a cada valor para achar seu código:

Decimal	**Cálculo ($+8$)**	**Binário**
-8 (mínimo)	$-8 + 8 = 0$	0000
-7	$-7 + 8 = 1$	0001
0	$0 + 8 = 8$	1000
$+1$	$1 + 8 = 9$	1001
$+7$ (máximo)	$7 + 8 = 15$	1111

- **Faixa de Valores ($n = 4$):** de -8 a $+7$.
- **Regra Geral:** de $-(2^{n-1})$ a $2^{n-1} - 1$.

Exercício 22: Excesso de 2^{n-1} $(n = 5)$

Para $n = 5$, o excesso é $2^{5-1} = 2^4 = \mathbf{16}$.

Regra: Some 16 ao número e converta o resultado para binário de 5 bits.

Converta para binário (5 bits, com Excesso de 2^{n-1}):

a) -15

b) -1

c) 0

d) 10

Gabarito: Exercício 22

a) -15: $-15 + 16 = 1 \rightarrow \mathbf{00001_2}$

b) -1: $-1 + 16 = 15 \rightarrow \mathbf{01111_2}$

c) 0: $0 + 16 = 16 \rightarrow \mathbf{10000_2}$

d) 10: $10 + 16 = 26 \rightarrow \mathbf{11010_2}$

Dica Visual

Repare que o zero (10000) divide a tabela exatamente no meio. Qualquer número menor que 16 começará com 0 (negativos), e qualquer número ≥ 16 começará com 1 (positivos).